SciNumTools DIPH5

Unified Scientific Contract and Experiment Record — Format Specification

Ondrej Pego Jaura

Contents

1 DIPH5 Version 3 — Normative Specification **2**
1.1 0. Status and conventions . . . 2
1.2 1. File identification . . . 2
1.3 2. Canonical top-level layout . . . 3
1.4 3. Model source archive . . . 3
1.4.1 3.1 `/model/sources` . . . 3
1.4.2 3.2 `/model/manifest` . . . 4
1.4.3 3.3 `/model/registrations` . . . 4
1.5 4. Configuration and overrides . . . 4
1.5.1 4.1 `/configuration/overrides` . . . 4
1.5.2 4.2 `/configuration/resolved` . . . 4
1.5.3 4.3 `/configuration/provenance` . . . 5
1.5.4 4.4 `/configuration/adapter_outputs` . . . 5
1.6 5. Execution record . . . 5
1.6.1 5.1 `/execution/command` . . . 5
1.6.2 5.2 `/execution/software` . . . 5
1.6.3 5.3 `/execution/runtime` . . . 5
1.6.4 5.4 `/execution/inputs` . . . 6
1.6.5 5.5 `/execution/outcome` . . . 6
1.7 6. Results registry . . . 6
1.7.1 6.1 Naming and identity . . . 6
1.7.2 6.2 Embedded numerical datasets . . . 6
1.7.3 6.3 External artifacts . . . 6
1.7.4 6.4 Diagnostics . . . 6
1.7.5 6.5 `/results/manifest` . . . 7
1.8 7. Lifecycle and API contract . . . 7
1.9 8. Mutability and integrity . . . 7
1.10 9. Versioning and compatibility . . . 7
1.11 10. Canonical hashing rules . . . 8
1.11.1 10.1 Canonical source manifest . . . 8
1.11.2 10.2 Resolved configuration fingerprint . . . 8
1.11.3 10.3 Result data hash . . . 9
1.12 11. Validation rules . . . 9
1.13 12. Interoperability test cases . . . 9
1.14 13. Deliberately out of scope for v3.0 . . . 10

2 Appendix: Legacy DIPH5 specification **10**
2.1 Scope . . . 10
2.2 Persisted environment boundary . . . 10
2.3 File identification and versioning . . . 10
2.3.1 Resolved adapter output plan (2.8) . . . 11
2.3.2 File naming . . . 11
2.4 Paths and objects . . . 11
2.4.1 Collections . . . 11
2.4.2 Collection and sibling order (2.10) . . . 12
2.5 Value datasets . . . 12

2.5.1 Value type table . . . 12
2.6 Node settings and provenance . . . 13
2.6.1 Source manifest . . . 13
2.6.2 Trace manifest . . . 13
2.7 Examples . . . 14
2.7.1 Scalar values and hierarchy . . . 14
2.7.2 Arrays and units . . . 14
2.7.3 Value groups . . . 15
2.7.4 Custom-unit manifest . . . 15
2.7.5 Ordered collections . . . 15
2.8 Compatibility . . . 15

1 DIPH5 Version 3 — Normative Specification

1.1 0. Status and conventions

This document specifies DIPH5 v3, the canonical on-disk format from SNT3 0.10.0 onward. The runtime reads and writes only v3 archives. It supports structural configuration archives, execution records, results, and a canonical resolved-configuration fingerprint for supported value types. Unsupported types, including platform-dependent `float128`, are marked unavailable.

DIPH5 v2 and lower are legacy, unsupported on-disk formats. The v2.11 specification is preserved in the appendix as a historical reference for reconstructing or migrating older archives with a pre-0.10.0 reader. The current runtime does not read or write those archives. References below to v2.11 node encoding describe the encoding retained inside v3; they do not imply support for v2 files.

MUST, **MUST NOT**, **SHOULD**, and **MAY** are normative. All HDF5 paths are absolute. UTF-8 means valid UTF-8; binary source and artifacts are stored as `uint8` arrays. Numeric metadata MUST use explicit fixed-width HDF5 types; strings SHOULD use variable-length UTF-8 strings except for byte-exact source content. Timestamps use RFC 3339 UTC strings with a `Z` suffix. SHA-256 hashes are lowercase hexadecimal of exact bytes, never of decoded text.

Design invariants: 1. A DIPH5 file always contains exactly one **resolved DIPL configuration** (the contract core). 2. An environment-only file is valid; execution and results are optional extensions **of the same schema**. 3. A v3 file describes one frozen effective configuration and at most one execution attempt. Repeated runs produce separate files. 4. The original source bytes, applied modifications and resolved parameter tree are different representations, not duplicates to be conflated. 5. The host application or external wrapper executes the scientific code and produces output. DIPH5 owns persistence, lifecycle, indexing, and integrity verification. 6. Results can be embedded or externally referenced. An external reference is never represented as if its bytes were archived. 7. The v2.11 node encoding is retained **within** `/configuration/resolved`, except for the explicit v3 changes in this document.

1.2 1. File identification

Required root attributes:

Attribute	Type	Value
`_DIPH5_Format`	UTF-8 scalar	`SciNumTools3 DIPH5`
`_DIPH5_Schema_Major`	uint32	`3`
`_DIPH5_Schema_Minor`	uint32	`0`
`_DIPH5_Schema_Patch`	uint32	`0`
`_DIPH5_Record_Id`	UTF-8 scalar	UUID, lowercase canonical form
`_DIPH5_State`	UTF-8 scalar	`prepared`, `running`, `completed`, `failed`, `aborted`
`_DIPH5_Created_At`	UTF-8 scalar	UTC timestamp
`_DIPH5_Profile`	UTF-8 scalar	`environment` or `experiment`

The profile is a **capability/intent indicator**, not a separate schema. `environment` means no `/execution` and no `/results`. `experiment` means `/execution` exists; `/results` MAY be absent or empty. An experiment record MUST have a `/execution` group. Readers MUST validate profile against group presence and reject inconsistencies. The format identity is not inferred from the extension.

All root groups listed below are format-owned; user DIPL names are scoped only beneath `/configuration/resolved`. Unknown root groups beginning with `_` are reserved; extensions SHOULD use `/extensions/<namespace>`.

1.3 2. Canonical top-level layout

```
/
├── model/                    # REQUIRED
│   ├── manifest/             # REQUIRED
│   ├── sources/              # REQUIRED; exact source bytes
│   └── registrations/        # REQUIRED; units, schemas/functions trace
├── configuration/            # REQUIRED
│   ├── overrides/            # REQUIRED; may be empty
│   ├── resolved/             # REQUIRED; v2.11-compatible node encoding
│   ├── provenance/           # REQUIRED
│   └── adapter_outputs/      # OPTIONAL; v2.8 resolved adapter plans
├── execution/                # OPTIONAL, required for experiment profile
│   ├── command/
│   ├── software/
│   ├── runtime/
│   ├── inputs/
│   └── outcome/
├── results/                  # OPTIONAL
│   ├── datasets/
│   ├── artifacts/
│   ├── diagnostics/
│   └── manifest/
└── extensions/               # OPTIONAL
```

An empty `/configuration/overrides` group means no modifications. An environment-only file MUST NOT contain `/results` or `/execution`. File writers MUST NOT create results for a configuration that has not been prepared and frozen.

1.4 3. Model source archive

1.4.1 3.1 `/model/sources`

Each contributing **base-model** source (including transitive imports, custom units/schemas/functions expressed in DIPL, and model source fragments) MUST be represented by a numbered group `/model/sources/000000`, `/000001`, etc., in stable source registration order. Applied override sources belong in `/configuration/overrides` and MUST NOT enter the base-model source manifest or its hash. Source names and paths are attributes, not HDF5 path components.

Each entry MUST contain:

Field	Storage	Description
`content`	1-D uint8 dataset	Exact bytes parsed, without text normalization
`source_id`	UTF-8 attr	Unique identifier within archive
`name`	UTF-8 attr	Original DIPL source identifier
`original_path`	UTF-8 attr	Original path, or empty for inline input
`parent_source_id`	UTF-8 attr	Importing source ID, or empty
`parent_line`	uint64 attr	1-based importing line, 0 if none
`sha256`	UTF-8 attr	SHA-256 of `content`
`role`	UTF-8 attr	`model`, `import`, `units`, `schema`, `function`, `other`
`encoding`	UTF-8 attr	`utf-8` for DIPL text

Repeated identical bytes MAY be deduplicated with HDF5 hard links to content datasets, but each source entry MUST preserve its own identity and provenance. Every referenced source ID MUST exist. Source collection order MUST NOT be inferred from HDF5 iteration order.

An archival writer MUST fail if a contributing source cannot be recovered exactly; it MUST NOT silently produce a source-complete archive with missing content. Sources extracted to disk MUST be confined to an explicitly selected output directory, using sanitized paths and collision handling; original absolute paths MUST NOT be used as extraction destinations.

1.4.2 3.2 `/model/manifest`

Required attributes: `model_id` (UUID), `source_count` (uint64), `model_sha256` (UTF-8), `dipl_language_version` (UTF-8), `producer_name` and `producer_version` (UTF-8).

`model_sha256` is the SHA-256 of the canonical source manifest encoding defined in §10; it identifies the archived model sources, not evaluated values. Optional metadata: title, description, authors, DOI, license.

1.4.3 3.3 `/model/registrations`

Preserve the v2.11 registration information: custom PUEL units in dependency order; trace identifiers for units, schemas, and functions; schema metadata and source positions. Use subgroups `units/` and `trace/` with numbered entries, retaining v2.11 field names and semantics. This is descriptive and interpretive metadata, **not serialized executable native callbacks**. A reader MUST register custom units before loading resolved quantities. Embedded DIPL source does not guarantee re-execution when custom native functions or external libraries are unavailable.

1.5 4. Configuration and overrides

1.5.1 4.1 `/configuration/overrides`

Each *applied* override has a numbered group in application order (`000000`, `000001`, …), including inline overrides. Each group MUST contain:

Field	Storage	Description
`content`	uint8 dataset	Exact applied override source bytes
`override_id`	UTF-8 attr	Stable identifier within this record
`name`	UTF-8 attr	Human-assigned name, or generated ID
`origin`	UTF-8 attr	`file`, `inline`, `generated`, `api`
`original_path`	UTF-8 attr	Optional origin path
`order`	uint64 attr	Contiguous, zero-based application order
`sha256`	UTF-8 attr	Hash of exact content
`source_id`	UTF-8 attr	Source archive ID if same content is registered there, otherwise empty

The applied override chain MUST be reproducible as source text and MUST preserve order. Override entries are **not** execution attempts. An override can be reused by many files/runs. Source-level override content is mandatory even if the resolved values are already stored. An override's source ID may refer to a source in `/model/sources`, but an override MUST NOT be counted as part of the *base model hash*. Inline overrides remain valid at the language/API level; they are materialized here with generated names.

1.5.2 4.2 `/configuration/resolved`

This group contains the evaluated DIPL hierarchy. **Within this group, retain v2.11 rules** for:

- `_DIPL_Kind`, `_DIPL_Path`, `_DIPL_Order`, map/list item metadata and `value_group`/`_DIPL_Value`.
- Scalar, array, null and UTF-8 value encodings; physical units and value types.
- Source, schema, condition, options, modification and override attributes.
- Local and remote link semantics, including unresolved remote links.

`_DIPL_Path` remains the *logical DIPL path* (for example `simulation.steps`), **not** `/configuration/resolved/simulation/steps`. Any existing code that assumes parameter nodes begin at HDF5 root MUST be updated to use this group as the serialization root. The existing v2.11 encoding and validation rules are normative except for relocation. Unsupported `float128` representations MUST produce an explicit error, not lossy conversion.

1.5.3 4.3 /configuration/provenance

Retain v2.11 evaluated dependency graph and modification provenance, including ordered modifications and source references. Required attributes on this group:

- configuration_sha256 — canonical resolved configuration fingerprint per §10.
- model_sha256 — equal to /model/manifest@model_sha256.
- prepared_at — UTC timestamp.
- evaluator_name, evaluator_version — UTF-8 strings.

All source references in resolved node metadata and provenance MUST resolve to an identity in /model/sources or /configuration/overrides, or be explicitly marked as unresolved legacy/imported references. A native v3 writer MUST NOT emit unresolved source identities. Override IDs and base-model source IDs occupy separate namespaces; a resolved override location may use the override's name as its source reference.

1.5.4 4.4 /configuration/adapter_outputs

Optional relocation of v2.11 /_DIPL_Outputs. Preserve ordered resolved mappings and all original v2.8 field semantics. These are **adapter input/output preparation plans**, not scientific results, and MUST NOT appear under /results.

1.6 5. Execution record

/execution exists only for the experiment profile. It is populated through the SNT3 lifecycle API, using information observed by the launcher or supplied by the host/scheduler. The host remains responsible for computation.

1.6.1 5.1 /execution/command

Required for a managed execution:

- argv: ordered 1-D UTF-8 dataset, **including argv[0]**, exactly as passed to the launch API.
- executable_invoked: UTF-8 attr; the program name/path as invoked.
- working_directory: UTF-8 attr; directory at launch.
- capture_origin: UTF-8 attr; launcher_observed, host_reported, imported.
- resolved_executable_path: optional UTF-8 attr.

A launcher command (srun, mpirun, shell, container runtime) and the scientific program are not necessarily the same. An optional /execution/command/launcher_argv ordered string dataset preserves the outer launch invocation. argv MUST identify which process it describes using command_role (scientific_program or outer_launcher). If the scientific child argv cannot be observed, do not invent it; store the observed launcher argv and mark child invocation unknown. Shell command strings MAY be stored additionally for display, but MUST NOT replace argument arrays.

Security: arguments can contain secrets. An archival policy MAY redact them, but MUST record argv_redacted=true and MUST NOT claim exact invocation preservation when redaction occurred.

1.6.2 5.2 /execution/software

Recommended: application_name, application_version, executable_sha256, git_commit, git_dirty, container_image_digest, and executable size. Hashes refer to the exact observed executable bytes; a path alone is not identity. Optional /execution/software/executable_bytes is a uint8 dataset containing the actual binary, with matching SHA-256. Optional dependency manifests may list dynamic libraries and hashes. Do not assume embedding one executable guarantees re-executability.

1.6.3 5.3 /execution/runtime

Optional groups requested/, observed/, system/, environment/. Store structured fixed-width fields where possible: CPU count, threads, MPI ranks, GPUs, memory limit, OS, architecture and scheduler job ID. requested values MUST NOT be presented as observed facts. Environment variables MUST use an explicit allowlist; credentials MUST NOT be archived by default. All runtime facts SHOULD carry capture origin and timestamp when ambiguity matters.

1.6.4 5.4 /execution/inputs

Each input artifact is a numbered entry with name, path_or_uri, storage (external or embedded), sha256 (when available), size_bytes (when available), format (optional), and verification (verified, unverified, missing). Embedded input bytes MAY be stored as content (uint8). An input path is not an input identity; content hashes are recommended.

1.6.5 5.5 /execution/outcome

Required attributes: status (prepared, running, completed, failed, aborted), created_at. Optional: started_at, finished_at, exit_code, termination_signal, message, capture_origin. The root _DIPH5_State MUST match outcome status. A retrospective import MAY have unknown timestamps or invocation details, but MUST mark provenance as imported and MUST NOT fabricate them.

State transitions: prepared -> running -> completed|failed|aborted. A collector MAY finalize a previously executed run via an explicit import operation that sets capture_origin=imported; it MUST preserve unknown fields. A terminal record is read-only through the normal API. A failed run MAY retain valid partial results.

1.7 6. Results registry

Results are scientific outputs, not DIPL input parameters. The host or collector registers them; DIPH5 validates and persists their descriptions.

1.7.1 6.1 Naming and identity

/results/datasets, /results/artifacts, and /results/diagnostics are optional groups; if /results exists, /results/manifest MUST exist. Each registered result MUST have a unique result_id (UUID), a name (human-facing UTF-8), a kind, a storage, a status (registered, writing, complete, failed, missing), and an explicit link to the owning execution via run_id. Result names are **not** used directly as HDF5 path components: use stable IDs or numbered entries to avoid slash/escaping collisions.

1.7.2 6.2 Embedded numerical datasets

Store at /results/datasets/<result_id>/data as an HDF5 dataset. Parent group required attributes:

- name, result_id, kind=dataset, storage=embedded, status, run_id
- logical_dtype (DIPL-compatible type name), units (optional PUEL canonical unit)
- dimensions (ordered UTF-8 string array; optional), description (optional)
- data_sha256 (required once finalized; defined over a canonical logical byte representation, §10)

Shape is determined by the HDF5 dataspace, not duplicated in metadata. Scalar datasets have scalar dataspace. Chunking, compression and HDF5 dimension scales MAY be used. If a result requires a type unsupported by the canonical hashing rules, the writer MUST reject finalized complete status rather than claim a hash it cannot reproduce.

1.7.3 6.3 External artifacts

Store at /results/artifacts/<result_id> with attributes name, result_id, kind=artifact, storage=external, status, run_id, path_or_uri, format (optional), size_bytes, sha256, verification. sha256 is over the **entire file bytes** and is required for a finalized verified artifact. Paths SHOULD be relative to the DIPH5 file's directory and MUST NOT be blindly trusted on extraction or resolution. A remote URI MAY be recorded; it does not guarantee future availability.

Embedded non-numeric artifacts MAY use storage=embedded with content (uint8) and corresponding byte hash. No mandatory copying of large scientific files is permitted.

1.7.4 6.4 Diagnostics

/results/diagnostics/<result_id> uses the same result identity fields. Diagnostics MAY be scalars, structured numeric datasets or references to logs/reports. A diagnostic MUST distinguish observed statistics (e.g. NaN counts) from validation judgments (e.g. physically invalid). A failed execution may have complete diagnostics.

1.7.5 6.5 `/results/manifest`

Required when `/results` exists. Attributes: `result_count` (uint64), `finalized` (uint8 boolean), `finalized_at` (optional UTC timestamp). A manifest table SHOULD index each result by ID, category, name, status and path. Every manifest entry MUST point to exactly one existing result; every registered result MUST appear exactly once. A completed run MUST have a finalized manifest, but zero results is permitted if the host explicitly finalizes an empty inventory.

Output contracts (declared expected result shapes/units/constraints in DIPL) are **future work**. v3 MUST NOT invent or require a new DIPL output-declaration syntax.

1.8 7. Lifecycle and API contract

A minimal API SHOULD support:

```
env = dip.load("model.dip", override=["accurate.dip"])
env.save("prepared.diph5")             # environment profile
exp = Experiment.from_environment("prepared.diph5", "run_001.diph5")
exp.prepare()                           # freeze model + resolved configuration
exp.start(command=[...], metadata={...})
exp.results.add("energy", 42.5, unit="J")
exp.results.attach("output.nc", name="output")
exp.complete(exit_code=0)
exp.verify()
```

These names are illustrative, not mandates for existing API compatibility.

- `prepare`: validate required sources, archive exact bytes, compute hashes, freeze configuration.
- `start`: create execution identity, capture command/runtime facts and set running status.
- `add`/`attach`: register results; `attach` SHOULD compute hash after external writer closes.
- `complete`: ensure execution is running, finalize result inventory, verify required hashes and mark completed. Exit code zero alone does not imply successful finalization.
- `fail`/`abort`: preserve partial results and record terminal outcome.
- `verify`: verify internal structural invariants, source hashes, embedded data and reachable external artifact hashes; return structured per-check status (`pass`, `fail`, `unavailable`), not a single misleading boolean.

The experiment file SHOULD NOT remain open throughout long HPC jobs by default. A wrapper may prepare, launch externally, and reopen for collection/finalization. One writer per file; no claim of multiwriter atomicity. SWMR is optional and does not implement transactions.

1.9 8. Mutability and integrity

The contract is immutable after `start`. In a `prepared` environment it MAY be edited only by reevaluating and regenerating its fingerprints. `complete`, `failed`, and `aborted` are terminal for normal APIs; a repair tool MUST create a new record ID or a separately audited revision, never silently rewrite scientific history.

HDF5 is not an immutable store and is not a transactional database. Hashes detect accidental or deliberate modification of covered data but are not authentication. A future digital-signature extension MAY authenticate a finalized manifest.

Writers SHOULD use flush/sync at lifecycle boundaries and provide crash recovery for incomplete files. A crash can leave an apparently `running` record; readers MUST NOT treat file closure as completion. Result checksum verification MUST operate on closed, stable external files.

1.10 9. Versioning and compatibility

v3 is intentionally incompatible at the on-disk root level with v2.11. v3 readers MUST reject unknown major versions. Unknown minor-version extensions MAY be ignored only when declared optional and when required core invariants can still be verified.

The runtime does not provide a v2.11 migration tool or reader. The v2.11 specification is retained to support possible future reconstruction. A v3 writer MUST NOT silently fabricate missing source or override content, and there is only one v3 schema.

1.11 10. Canonical hashing rules

Hashing needs deterministic, testable encodings, not a vague "hash the HDF5 group".

1.11.1 10.1 Canonical source manifest

For each model source in ascending numbered-entry order, form UTF-8 JSON objects with keys `source_id`, `name`, `parent_source_id`, `parent_line`, `role`, `sha256`, sorted lexicographically; serialize compact JSON (`ensure_ascii=False`, no insignificant whitespace, normalized integer notation), and UTF-8 encode. Hash the UTF-8 bytes of the JSON array. Source content hashes remain SHA-256 of raw bytes. The manifest hash MUST NOT include filesystem path or HDF5 object addresses.

1.11.2 10.2 Resolved configuration fingerprint

For DIPH5 3.0, `configuration_sha256` hashes the encoding defined below. Its rules are part of DIPH5 3.0 and have no separate format version. The encoding covers each node's logical DIPL path, kind, value type, shape, unit spelling, value bytes, link identity, constraints, and value-origin and modification data. Floating types without a portable encoding, including platform-dependent `float128`, are unavailable for fingerprinting. Unit spelling follows PUEL normalization.

The fingerprint covers the model manifest hash, ordered override source hashes, and resolved node manifest. It MUST exclude timestamps, execution details, results, and HDF5 layout/chunking/compression.

The SHA-256 input is the concatenation below. Counts and lengths are unsigned 64-bit little-endian integers; text is UTF-8 prefixed by its byte length. There are no terminators or padding.

1. ASCII `diph5-3.0`, followed by the model-manifest SHA-256 as text.
2. The number of ordered overrides, followed by each override SHA-256 as text in override order.
3. The resolved group, encoded recursively as follows.

A group consists of its child count followed by its children sorted by their UTF-8 HDF5 names. Each child is the ASCII tag `g` for a group or `d` for a dataset, its name as text, its attributes, and its group or dataset contents. The names correspond to validated DIPL paths; `_DIPL_Value` denotes a value-group payload.

Attributes are sorted by name and begin with their count. Each attribute consists of its name as text and one of: `s` followed by scalar text; `l` followed by a string count and each string as text; or `u` followed by an unsigned 64-bit integer. The `created` and `modified` attributes and their `metadata_present` entries are excluded; if no presence entries remain, omit the presence attribute. Attribute storage width and creation order do not affect the stream. Unrecognized attribute types are unavailable for fingerprinting.

A dataset consists of `n` for a null dataspace, or `v`, the rank, each extent, and its values in C order. Integers use their declared width in little-endian, with two's-complement signed integers. Float32 and float64 use their IEEE binary32 or binary64 bits in little-endian, preserving NaN payloads and signed zero. Strings are individually length-prefixed. The dtype and node semantics appear among the attributes. HDF5 chunking and compression do not affect the stream.

The following vectors use exact source text including the final newline and the current SNT source-identity convention:

Source	SHA-256
`answer int = 7\n`	`b75c824d5e69cbbb6abed9f33adfc503cd0c347969087746044eb24dff64dc`
`a int8[2] = [-128, 127]\nb float32 = -0.0\nc str = "µ"\n`	`a6fbef7eca599dacb5ae4aa9bb59269a2c1ed146fe409da84238573743d7bb`
`physics\n  radius float64 = 4 m\n  limit int16 = 6\ncatalog map\ncatalog[alpha]\n  label str = "x"\n`	`70e39d758705403cace4b6c6c7c4a36b1f46a4d548950b39ba23d526f65a4b`

Writers set `fingerprint_status` to `pass` and store the lowercase digest in `configuration_sha256` when every resolved value can be encoded. Readers recompute and compare that digest; a mismatch fails integrity verification. For an unsupported value type, writers set `fingerprint_status` to `unavailable` and leave the digest empty. The current SNT3 encoder also marks an individual resolved dataset unavailable when its decoded payload exceeds 256 MiB; this is an implementation resource limit, not a change to the canonical encoding. The independent reference vectors above exercise the portable encoding.

1.11.3 10.3 Result data hash

For numeric results, use the same canonical logical type, shape and value-byte encoding as above, prefixed by `diph5-result-v1`; include units and dimension names. For embedded binary artifacts and external files, SHA-256 hashes exact bytes. Different HDF5 compression or chunking MUST NOT change the logical dataset hash.

The implemented numeric result encoding is the following concatenation, with no padding or terminating NUL:

1. The ASCII bytes `diph5-result-v1`.
2. The length-prefixed UTF-8 DIPL logical dtype, followed by the rank as an unsigned 64-bit integer and each extent as an unsigned 64-bit integer.
3. The length-prefixed UTF-8 units string, the number of dimension names as an unsigned 64-bit integer, and each dimension name as a length-prefixed UTF-8 string.
4. Each value in C order, using the logical dtype's width: two's-complement signed integers, unsigned integers, or IEEE 754 binary32/binary64. Boolean values use one unsigned byte and character values use one signed byte.

All integers and floating-point bits are little-endian. Each string length is an unsigned 64-bit byte count; an absent units string and an absent dimension list are encoded with zero length and zero count. The encoding uses the stored units spelling. For example, a `float64` 2×2 array `[1, 2, 3, 4]` with units `m` and dimension names `row`, `column` hashes to `cccf3bbbfabfcd462a458786e248fb8c1dd44a9c4a429c0d73cb64038a9cca5a`. SNT3 supports `bool`, `char`, signed and unsigned integers of 8, 16, 32 and 64 bits, and `float32` and `float64`. Portable `float128` result hashing and string results are not yet implemented and are rejected.

Additional golden vectors use shape `[2]`, no units, and no dimension names:

Dtype	Values	SHA-256
`int8`	`[-128, 127]`	`c4f9f86b002092ad9a988fbadea14cf8295cf434434a`
`uint64`	`[0, 18446744073709551615]`	`dc49b5b6538e8d07a52a74bb8cbe24ec73bc42dddcdd`
`float32`	`[1.5, -2.25]`	`c653b9cecb96ab3d43901dcf9cd7e52e281e8b4b0f59`
`float64`	`[1.5, -2.25]`	`33513fb83f8295b28b20fe7d7028cd07b00cc82f88b9`
`bool`	`[0, 1]`	`95ec3f3b94e79f708efb8499c1debf260fdc009b5243`

The canonical encoder covers only the supported portable value types defined above. For unsupported types such as platform-dependent `float128`, writers MUST mark the configuration fingerprint unavailable and MUST NOT claim a portable digest. The golden vectors above define the portable cases.

1.12 11. Validation rules

A conforming validator MUST check: - Root identity, supported version, required groups and profile/state consistency. - Source and override numbering, order, IDs, references and SHA-256 hashes. - Exactly one resolved hierarchy and correct v2.11 node-kind/path/order semantics. - Custom-unit registration order and resolvable unit encodings. - Consistency of model and configuration hashes. - No execution/results for environment profile. - Execution outcome status equals root status; terminal timestamps are valid when present. - Command argument arrays and capture origin; never infer unknown invocations. - Unique result IDs, complete manifest inventory, allowed statuses, data type/shape consistency. - Embedded result checksums; external artifacts classified as verified/unverified/missing. - Completed run has finalized result manifest. - No unsafe automatic path traversal when extracting sources or artifacts.

Verification output MUST distinguish **structural validity**, **integrity validity**, **external artifact availability**, and **re-executability**. Passing the first two does not imply that the experiment can be rerun.

1.13 12. Interoperability test cases

1. Round-trip scalar, arrays, maps, lists, value-groups, nulls, links, ordered children, custom units, metadata and dependencies from v2.11 semantics under the new root.
2. Archive multi-file imports, identical-content sources, anonymous and named overrides, multiple ordered overrides; byte-exact extraction.
3. Reject missing source bytes, incorrect hashes, duplicate IDs, broken source references, invalid UTF-8 metadata, invalid node order.
4. Environment-only write/read with no execution/results.
5. Prepared -> running -> completed; running -> failed/aborted; reject invalid transitions.

6. Exact argv with whitespace, quotes, empty strings, Unicode, launcher vs child and redaction flags.
7. Embedded scalar/array results; external NetCDF artifact; missing artifact; corrupt artifact; partial result after failure.
8. Hash invariance across HDF5 compression/chunking and round-trip; golden vectors on Linux/macOS/Windows.
9. Interrupted writer and stale running file; verify incomplete state is never reported completed.
10. Reject v2.11 archives clearly in the v3 reader.
11. Path traversal and untrusted HDF5 content tests; bounds on source/result count and archive size.
12. Validate old readers reject v3 clearly; new readers load all v3 profiles through the same schema implementation.

1.14 13. Deliberately out of scope for v3.0

- Job scheduling, workflow DAGs, MPI orchestration, container execution.
- Multiple runs or mutable override libraries in one experiment file.
- A new DIPL output-declaration language.
- Automatic capture of every dynamic library, subprocess, or environment variable.
- Guarantee of bitwise numerical reproducibility.
- Replacing NetCDF, HDF5 scientific outputs, or existing model-native formats.
- Digital signatures, global experiment catalogs, and network synchronization.

2 Appendix: Legacy DIPH5 specification

This appendix preserves the v2.11 on-disk specification for historical interpretation and migration. Its requirements describe legacy archives only; SNT3 0.10.0 and later read and write only the v3 format specified above.

2.1 Scope

DIPH5 v2.11 was the HDF5 interchange format used by earlier SciNumTools releases to persist an *evaluated DIPL environment*. It stores evaluated value nodes together with their DIPL paths, collection structure, units, settings, and provenance. It also stores link nodes as source and target paths without copying linked data.

The persisted object is therefore a data environment suitable for exchange with scientific applications. It is not a serialization of the complete parser or evaluation runtime.

This document specifies the on-disk representation. It does not define the DIPL language itself; the DIPL specification defines the language concepts represented here.

2.2 Persisted environment boundary

The file contains the evaluated value-node tree and the node-level information needed to interpret and trace those values. In particular, node settings, units, schemas, source information, and provenance are persisted as dataset attributes.

Implementation-specific runtime registries are not serialized as executable objects. This includes global function definitions, branching state, and the complete unit and schema registries. DIPH5 version 2 additionally stores a source manifest containing source identities, paths, parent relationships, and content fingerprints. Version 2.1 also stores durable identifiers for registered units, schemas, and functions. It does not embed complete source text, parsed source nodes, or executable functions. Version 2.2 adds `value_group` objects. Version 2.3 persists custom PUEL unit registrations required to interpret evaluated quantities. Version 2.4 adds descriptive schema metadata and source locations to schema trace entries. Version 2.5 records which evaluated values were overridden and where their replacement values originated. Version 2.6 retains applied value modifications and the parent table and original column order for table columns. A loaded DIPH5 file must therefore be treated as an evaluated environment, not as a source from which the original DIPL program can be reconstructed exactly.

Version 2.7 retains the evaluated dependency graph. Version 2.8 may retain the resolved output plan of an application adapter. Output plans contain values and decisions, not executable adapter callbacks. Version 2.9 may retain display labels for value options. Version 2.10 records explicit sibling order for hierarchy children and map entries. Version 2.11 stores identity-preserving link targets without embedding the linked data.

2.3 File identification and versioning

The HDF5 root object MUST contain the following scalar attributes:

Attribute	Type	Required value
`_DIPL_Format`	UTF-8 string	`SciNumTools3 Environment`
`_DIPL_Schema_Version`	unsigned integer	`2`
`_DIPL_Schema_Version_Minor`	unsigned integer	`11`

Readers MUST reject files with a different format identifier or unsupported schema version. Version 2.11 readers support version 1 files and versions 2.0 through 2.11. Future schema revisions MUST preserve the meaning of existing attributes or increment the major or minor schema version.

2.3.1 Resolved adapter output plan (2.8)

An adapter-run snapshot MAY contain a root `_DIPL_Outputs` group. Each numbered child group, in declaration order, stores one resolved mapping. Its attributes are `id`, `target`, `key`, `source_path`, `rule`, `origin`, `replacements`, `dependencies`, `active`, `value_path`, `_DIPL_Node_Type`, and `_DIPL_Value_Type`; `units` is present for values with units. The `value` dataset uses the same typed HDF5 value encoding as parameter datasets. A missing `_DIPL_Outputs` group means the snapshot has no retained plan, so older adapters may derive their output policy from the loaded environment. These mappings are not DIPL parameter nodes and MUST NOT appear in normal environment selection.

2.3.2 File naming

Files SHOULD use the `.diph5` filename extension to make their purpose clear. The extension is only a naming convention and is not normative. Readers MUST identify a file from the root `_DIPL_Format` and `_DIPL_Schema_Version` attributes, regardless of its filename.

2.4 Paths and objects

Each component of a fully qualified DIPL path is represented by an HDF5 path component. A value node without child nodes is an HDF5 dataset at its resolved path. Intermediate groups represent DIPL hierarchy and carry `_DIPL_Kind=group` and `_DIPL_Path` attributes.

The `_DIPL_Path` attribute stores the corresponding fully qualified DIPL path and is the authoritative mapping back to DIPL semantics.

2.4.1 Collections

Collection containers and items are represented by groups. Their meaning is identified by `_DIPL_Kind`:

`_DIPL_Kind`	Meaning	Additional attributes
`map`	keyed collection	—
`map_item`	item of a keyed collection	`_DIPL_Key`, `_DIPL_Order`
`list`	ordered collection	—
`list_item`	item of an ordered collection	`_DIPL_Index`

Collection item groups may contain nested groups and value datasets. They are containers and cannot themselves represent DIPL value nodes.

The complete object-kind table is:

HDF5 object	`_DIPL_Kind`	`_DIPL_Path`	Other required attributes
hierarchy group	`group`	group path	—
keyed collection	`map`	collection path	—
keyed item	`map_item`	item path	`_DIPL_Key`
ordered collection	`list`	collection path	—
ordered item	`list_item`	item path	`_DIPL_Index`
value with children	`value_group`	value path	`_DIPL_Value` dataset
value dataset	`value`	value path	`_DIPL_Node_Type`, `_DIPL_Value_Type`

An implementation MUST use the kind attribute to distinguish a collection from an ordinary hierarchy group. HDF5 object names alone are not sufficient, because a numbered or keyed collection item is deliberately flattened into a normal group path.

2.4.2 Collection and sibling order (2.10)

Version 2.10 adds the `_DIPL_Order` unsigned integer attribute. A writer MUST store it on each direct child object of a hierarchy group or collection item, and on every map item group. It records a zero-based position among that parent's named children. The attribute is written on child groups and value datasets, including value groups. Positions within a parent MUST be unique and contiguous, beginning at zero. The attribute is optional only when reading older files or when the object is a list item.

Map item groups also carry `_DIPL_Order`, recording map declaration order. List item groups continue to use `_DIPL_Index`; readers MUST order list items by its numeric value and MUST ignore `_DIPL_Order` for list ordering. An override of an existing map or group member retains its position; a newly introduced member is appended according to the evaluated environment order.

Readers MUST honor a complete `_DIPL_Order` sequence. If the attribute is absent or incomplete for a set of siblings, readers use lexical HDF5 object name order as a deterministic fallback. Files written before version 2.10 cannot recover declaration order for maps, groups, or group children; list order remains recoverable from `_DIPL_Index`. Keyed lookup semantics are unaffected by enumeration order.

2.5 Value datasets

Every value dataset has `_DIPL_Kind=value`, `_DIPL_Path`, `_DIPL_Node_Type`, and `_DIPL_Value_Type` attributes. `_DIPL_Array` identifies array values; scalar values use a scalar dataspace and arrays use a simple dataspace whose dimensions match the DIPL value shape.

A link node is stored on the hierarchy group at its requested path with `_DIPL_Node_Type=link` and the required UTF-8 string attributes `_DIPL_Link_Source` and `_DIPL_Link_Target`. The group retains its normal hierarchy attributes, such as the map key or list index when the link is a collection item. An empty source means the local DIPH5 environment. The target data is stored only at its canonical path. Loading a snapshot preserves remote link identity but does not resolve it because named source definitions are not restored; traversal through such a link MUST report an unresolved remote link without making snapshot loading fail. Local links resolve against the nodes in the snapshot.

Values are written using native HDF5 numeric types. Strings use fixed-width UTF-8 HDF5 string datatypes. A null DIPL value is represented by a dataset with an HDF5 null dataspace.

2.5.1 Value type table

The `_DIPL_Value_Type` value identifies the logical DIPL type. The following table lists the concrete types currently written by DIPH5 and their HDF5 storage types:

`_DIPL_Value_Type`	Logical value	HDF5 datatype	Scalar/array
`bool`	Boolean	native unsigned 8-bit integer	both
`char`	Character	native signed 8-bit integer	both
`int8`, `int16`, `int32`, `int64`	Signed integer	matching native signed integer	both
`uint8`, `uint16`, `uint32`, `uint64`	Unsigned integer	matching native unsigned integer	both
`float32`	Single precision real	native 32-bit float	both
`float64`	Double precision real	native 64-bit float	both
`float128`	Extended precision real	native long double	both
`str`	UTF-8 string	fixed-width UTF-8 string	both

For a scalar, the dataset has an HDF5 scalar dataspace. For an array, the dataset has a simple dataspace and `_DIPL_Array` is non-zero. The dimensions are stored in the dataspace rather than in a separate attribute.

2.6 Node settings and provenance

When present, node settings are stored as attributes on the value dataset:

`_DIPL_Constant`, `_DIPL_Condition`, `_DIPL_Format_Spec`, `_DIPL_Tags`, `_DIPL_Options`, `_DIPL_Option_Units`, `_DIPL_Option_Labels`, `_DIPL_Schemas`, and `_DIPL_Value_Origin`.

Version 2.9 adds the optional `_DIPL_Option_Labels` UTF-8 string array. When present, it has one entry for each `_DIPL_Options` entry in the same order. When absent, options have no display labels. Labels do not affect value validation.

Version 2.5 adds `_DIPL_Override` (unsigned 8-bit boolean) to each value dataset. If true, `_DIPL_Override_Source`, `_DIPL_Override_Line`, and `_DIPL_Override_Code` identify the modification that supplied the effective value. The ordinary source attributes continue to identify the declaration.

Version 2.6 adds `_DIPL_Modification_Sources`, `_DIPL_Modification_Lines`, and `_DIPL_Modification_Codes` as equal-length UTF-8 string arrays. They record applied value modifications in evaluation order; line numbers are decimal strings. Ignored changes are not included. The declaration and effective override remain in their existing attributes.

Version 2.6 also adds `_DIPL_Table_Path` (UTF-8 string) and `_DIPL_Table_Column_Index` (unsigned integer) to datasets produced from a table declaration. The path identifies the parent DIPL table, and the index preserves its column order. Both attributes are absent on ordinary value datasets. Older files remain readable, but they do not retain table identity.

Source and provenance information uses `_DIPL_Source`, `_DIPL_Source_Line`, `_DIPL_Source_Code`, and the metadata attributes `description`, `authors`, `title`, `journal`, `year`, `volume`, `issue`, `pages`, `doi`, `url`, `version`, `created`, `modified`, `license`, `rationale`, `recommended_range`, `performance_impact`, `scientific_impact`, `deprecated`, `replacement`, `since`, `category`, and `visibility`. The plural metadata attributes `native`, `requires`, `conflicts`, `implies`, `see`, and `example` are UTF-8 string-array attributes. Physical units are stored in the `units` attribute using their canonical PUEL representation.

2.6.1 Source manifest

Version 2 files contain a root `/_DIPL_Sources` group. Each child is a numbered source-manifest entry so DIPL source names never need to be encoded as HDF5 object names. Each entry has the following attributes:

Attribute	Type	Meaning
`_DIPL_Source_Name`	UTF-8 string	Named DIPL source identifier.
`_DIPL_Source_Path`	UTF-8 string	Path recorded when the source was parsed, or a path relative to the DIPH5 file's directory when saved with the relative source path policy; it may be empty for inline input.
`_DIPL_Source_Parent`	UTF-8 string	Parent source identifier, when one exists.
`_DIPL_Source_Parent_Line`	unsigned integer	Parent source line associated with the registration.
`_DIPL_Source_Hash_Algorithm`	UTF-8 string	Digest algorithm, currently `SHA-256`.
`_DIPL_Source_Hash`	UTF-8 string	Lowercase hexadecimal digest of the exact UTF-8 bytes parsed by DIP.

The manifest is an integrity and provenance aid, not a source archive. A consumer can hash an available source file and compare the result with the stored digest, but loading DIPH5 does not recreate executable source definitions or enable source-qualified lookups. `_DIPL_Sources` is reserved at the DIPH5 root; a DIPL environment with that top-level path cannot be saved.

2.6.2 Trace manifest

Version 2.1 files contain a root `/_DIPL_Trace` group. Each child is a numbered trace-manifest entry. It records the stable internal identifier assigned during parsing for every registered unit, schema, and function:

Attribute	Type	Meaning
`_DIPL_Trace_Id`	UTF-8 string	Internal trace identifier, for example `DIP0_UNIT0`.
`_DIPL_Trace_Name`	UTF-8 string	Registered public name.
`_DIPL_Trace_Kind`	UTF-8 string	`unit`, `schema`, `function_value`, or `function_nodes`.

The trace manifest preserves diagnostic and documentation identity only. It does not recreate a custom unit definition, schema nodes, or an executable function on load. `_DIPL_Trace` is reserved at the DIPH5 root; a DIPL environment with that top-level path cannot be saved.

In version 2.4, schema trace entries additionally carry `_DIPL_Schema_Source` (the source-manifest name), `_DIPL_Schema_Source_Line` (unsigned declaration or first-body line), and the same optional metadata attributes used by value nodes, including `description`, `authors`, and `doi`. These fields describe the schema and its origin; they do not provide a reusable definition. Earlier files retain schema IDs and names but have no schema-level metadata or source location. Schema-derived value datasets also carry `_DIPL_Node_Schema_Id`, which refers to the schema trace ID that supplied the node. Group and collection schema associations remain in `_DIPL_Schemas` attributes on their HDF5 groups.

2.7 Examples

2.7.1 Scalar values and hierarchy

This DIPL fragment:

```
simulation
  steps int = 100
  enabled bool = true
```

is represented as:

```
/simulation                      group   (_DIPL_Kind=group)
/simulation/steps                dataset (_DIPL_Kind=value,
                                          _DIPL_Path=simulation.steps,
                                          _DIPL_Value_Type=int32)
/simulation/enabled              dataset (_DIPL_Kind=value,
                                          _DIPL_Path=simulation.enabled,
                                          _DIPL_Value_Type=bool)
```

`/simulation/steps` has a scalar dataspace and a native signed 32-bit HDF5 datatype. `/simulation/enabled` has a scalar dataspace and a native unsigned 8-bit HDF5 datatype.

2.7.2 Arrays and units

This DIPL value:

```
boundary[inlet]
  velocity float[3] = [1.0, 0.0, 0.0] m/s
```

is represented as:

```
/boundary                        group   (_DIPL_Kind=map,
                                          _DIPL_Path=boundary)
/boundary/inlet                  group   (_DIPL_Kind=map_item,
                                          _DIPL_Path=boundary[inlet],
                                          _DIPL_Key=inlet)
/boundary/inlet/velocity         dataset (_DIPL_Kind=value,
                                          _DIPL_Path=boundary[inlet].velocity,
                                          _DIPL_Value_Type=float64,
                                          _DIPL_Array=1,
                                          units=m*s-1)
```

The velocity dataset has rank one and dimension `3`.

2.7.3 Value groups

When a DIPL value node has descendants, its resolved HDF5 path is a group rather than a dataset. The group has `_DIPL_Kind=value_group` and `_DIPL_Path` set to the value node's fully qualified path. Its own value is stored in a reserved child dataset named `_DIPL_Value`; that dataset uses the normal `value` dataset attributes, including the real DIPL path rather than a synthetic sentinel path. All other children of the group are regular DIPL descendants.

```
feature bool = true
  setting int = 2
```

is represented as:

```
/feature                          group   (_DIPL_Kind=value_group,
                                           _DIPL_Path=feature)
/feature/_DIPL_Value              dataset (_DIPL_Kind=value,
                                           _DIPL_Path=feature,
                                           _DIPL_Value_Type=bool)
/feature/setting                  dataset (_DIPL_Kind=value,
                                           _DIPL_Path=feature.setting,
                                           _DIPL_Value_Type=int32)
```

`_DIPL_Value` is format-owned and is not a DIPL child node. A direct DIPL child with that name beneath a value group is invalid and writers MUST reject it. The payload is a value node and therefore MUST NOT carry applied schemas; schemas are represented only by their expanded group or collection-item structure.

2.7.4 Custom-unit manifest

DIPH5 2.3 stores custom PUEL registrations in `/_DIPL_Units`. Each numbered entry carries `_DIPL_Unit_Name`, `_DIPL_Unit_Definition`, `_DIPL_Unit_Order`, and `_DIPL_Trace_Id`. Readers MUST register entries in ascending order before reading value datasets, so definitions may depend on earlier custom units. `_DIPL_Units` is format-owned and cannot be a top-level DIPL path.

2.7.5 Ordered collections

This DIPL fragment:

```
samples[]
  time float = 0 s
samples[]
  time float = 1 s
```

is represented by a `list` group containing `list_item` groups:

```
/samples                          group   (_DIPL_Kind=list,
                                           _DIPL_Path=samples)
/samples/0                        group   (_DIPL_Kind=list_item,
                                           _DIPL_Path=samples[0],
                                           _DIPL_Index=0)
/samples/0/time                   dataset (_DIPL_Path=samples[0].time)
/samples/1                        group   (_DIPL_Kind=list_item,
                                           _DIPL_Path=samples[1],
                                           _DIPL_Index=1)
/samples/1/time                   dataset (_DIPL_Path=samples[1].time)
```

The `_DIPL_Index` attribute, rather than HDF5 group iteration order, defines the order of list items.

2.8 Compatibility

Implementations SHOULD preserve unknown attributes when editing a DIPH5 file and MUST ignore attributes they do not understand. They MUST reject a file whose format identifier or schema version is unsupported rather than silently interpreting it as a different format.